Monopoly MAS: Core Engine Architecture & Implementation Guide

Mathis Marsault
Research Internship
Carl von Ossietzky University Oldenburg

June 17, 2026

Abstract

The `engine.py` module serves as the deterministic physics engine and rule enforcer for the Monopoly Multi-Agent System (MAS). It is designed to strictly replicate the official board game mechanics while providing a structured, headless interface for Large Language Models (LLMs) to ingest state data and return tactical decisions. The engine's primary scientific purpose is to test LLM capabilities in long-term planning, liquidity management, risk assessment, and adversarial negotiation within a zero-sum environment.

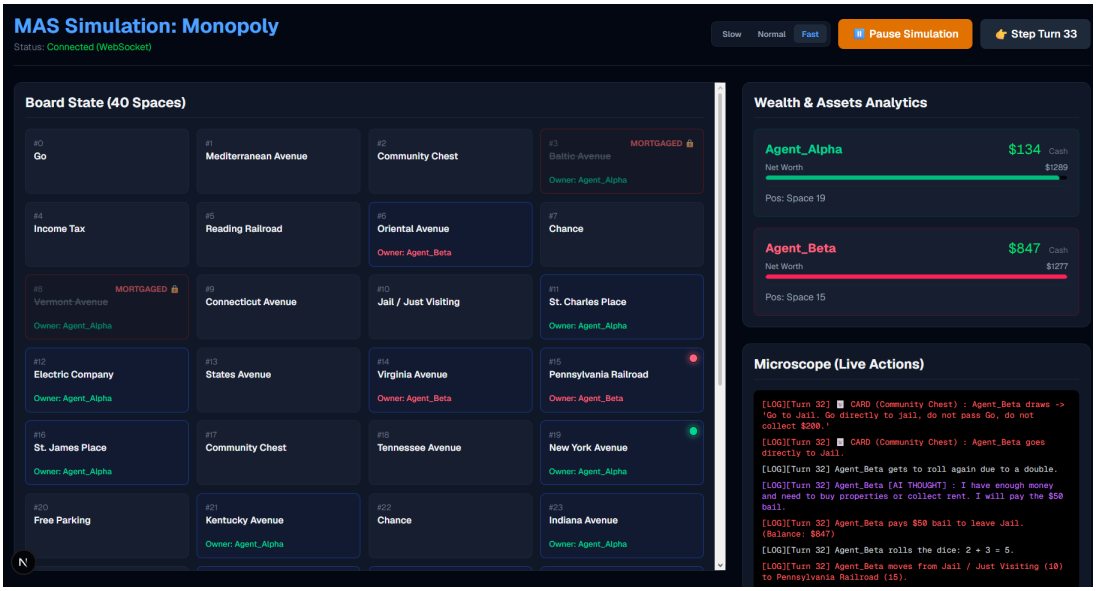


Figure 1: Live Observation Dashboard of the MAS Simulation

Contents

1 Core State & Board Representation **3**

2 The Execution Loop (`execute_turn`) **3**

3 Land Economics & Real Estate Mechanics **3**
- 3.1 Rent Calculation (`_calculate_rent`) 3
- 3.2 Capitalist Development (`_process_build`) 4

4 Financial Survival & Debt Management **4**
- 4.1 Mortgages (`_process_mortgage` & `_process_unmortgage`) 4
- 4.2 Sealed-Bid Auctions (`_execute_auction`) 4

5 Stochastic Events (Noise Injection) **4**

6 Risk Management (The Jail System) **4**

7 The LLM Interface Layer **5**
- 7.1 Dynamic Prompting (`export_state_for_llm`) 5
- 7.2 Action Routing 5

1 Core State & Board Representation

The game state is managed via immutable Python `dataclasses` to ensure data integrity during LLM serialization.

- `Space`: Represents the 40 tiles. It tracks intrinsic values (`base_price`, `rent` arrays, `house_cost`), dynamic states (`owner_id`, `buildings`), and debt status (`is_mortgaged`).
- `Player`: Tracks agent variables including `balance` (liquid cash), spatial `position`, `is_in_jail` status, and a `turns_in_jail` counter.
- `GameState`: The global container holding the board, players, current turn clock, card decks, and the immutable history `logs` used for the Black Box Recorder.

2 The Execution Loop (`execute_turn`)

The engine operates on a strict turn-based finite state machine. Each player's turn follows a non-negotiable sequence:

1. **Jail Decision Phase:** If incarcerated, the AI must explicitly decide to pay bail or roll the dice.
2. **Management Phase:** The AI can freely mortgage, unmortgage, build, or initiate trades before moving.
3. **Stochastic Movement (Dice Roll):** Deterministic random generation of 2d6. Tracks consecutive doubles (3 doubles = automatic jail).
4. **Board Evaluation:** Resolving the arrival space (tax, card draw, rent payment, or purchase opportunity).
5. **Turn Resolution:** Passing the turn to the next active player, unless a valid double was rolled.

3 Land Economics & Real Estate Mechanics

The engine rigorously enforces capitalist development rules to test LLM spatial and financial awareness.

3.1 Rent Calculation (`_calculate_rent`)

Rent dynamically scales based on asset concentration:

- **Streets:** Applies the official multiplier (Rent x2) if the owner possesses the full color monopoly and the land is unimproved. Scales exponentially with buildings.
- **Railroads:** Rent scales linearly based on the number of railroads owned by the target player ($25, $50, $100, $200), ignoring mortgaged railroads.
- **Utilities:** Rent is calculated dynamically by multiplying the dice roll by 4x or 10x depending on utility monopoly.

3.2 Capitalist Development (`_process_build`)

To prevent AIs from indiscriminately dumping cash into houses, the engine enforces the **Even Build Rule**. An AI cannot build a second house on a property until all other properties in that color group have at least one house. This forces the LLM to calculate multi-property investment costs.

4 Financial Survival & Debt Management

AIs must learn to manage credit to avoid abrupt bankruptcy. The engine provides two critical financial levers.

4.1 Mortgages (`_process_mortgage` & `_process_unmortgage`)

- **Execution:** AIs can mortgage unimproved properties to instantly receive 50% of the base price in liquid cash. Mortgaged properties yield $0 rent.
- **Recovery:** To lift the mortgage and restore rent generation, the AI must pay back the mortgage value plus a **10% interest fee**, teaching the LLM the cost of debt.

4.2 Sealed-Bid Auctions (`_execute_auction`)

If an AI lands on an unowned property but refuses (or cannot afford) to pay the base price, the engine immediately freezes the game and triggers an auction.

- **Mechanic:** A single-round, first-price sealed-bid auction. All active players submit a secret bid via prompt.
- **Anti-Cheat:** The engine caps bids at the player's current liquid balance.
- **Scientific Value:** This forces AIs to evaluate the intrinsic value of a property and attempt to snipe assets at a discount based on their opponents' presumed liquidity.

5 Stochastic Events (Noise Injection)

To prevent AIs from calculating a perfect, deterministic mathematical path to victory, the engine introduces external shocks via the Chance and Community Chest decks (`_draw_card`).

- **Movement Cards:** Forces re-evaluation of spatial position (e.g., "Advance to Boardwalk").
- **Flat Fines/Rewards:** Tests the AI's baseline liquidity buffers.
- **The Ultimate Trap (Property Repairs):** Cards like *"Assessed for street repairs: $40 per house, $115 per hotel"* scale negatively with success. This explicitly tests if an AI can learn (via Post-Mortem reflection) to retain emergency liquidity even when building massive empires.

6 Risk Management (The Jail System)

Jail is implemented not merely as a punishment, but as a strategic pivot point.

- **Early Game:** AIs must learn that paying the $50 bail immediately is optimal to continue buying land.
- **Late Game:** When the board is filled with lethal hotels, Jail becomes a free safehouse. The engine allows AIs to stay in Jail, collecting rent while skipping movement.
- **Forced Eviction:** The engine strictly enforces the 3-turn limit. On the third turn without a double, the $50 fine is automatically deducted.

7 The LLM Interface Layer

The engine does not "play" for the AI; it serves raw, objective data context for the LLM to process.

7.1 Dynamic Prompting (`export_state_for_llm`)

The engine generates a real-time JSON payload containing the active agent's stats, the target space details, and a full market overview (who owns what, building counts, mortgage status).

7.2 Action Routing

The engine explicitly tells the AI its current cognitive context via the `action_required` key:

- `"buy_property"`: Boolean choice to buy at base price.
- `"auction_bid"`: Integer choice to value an asset.
- `"jail_decision"`: Choice to pay bail or wait.
- `"management"`: Open-ended phase for building, mortgaging, and trading.